

APPCRONS

Backend API

Product Specification Document

Version 1.0 — April 14, 2026

Repository: github.com/Tibz-Dankan/appcrons

Prepared for: TestSprite Automated Test Generation

TestSprite Directives

- Use real test fixtures — no placeholder values (e.g. use "alice.nakato@gmail.com", not "user@example.com").
- Every endpoint must include ALL edge cases listed in its section.
- Auth edge cases (missing token, expired token, wrong user) must be tested on every protected endpoint.
- Assert both HTTP status code AND response body fields in every test.

1. Product Overview

Appcron is a backend service that keeps free-tier Render.com backend instances alive by sending automated HTTP pings at scheduled intervals. It prevents the 15-minute idle-shutdown behaviour of Render's free tier, which would otherwise force users' servers into a cold-start state lasting up to 50 seconds.

Property	Value
Language	Go (Golang 1.23)
Framework	Standard net/http + gorilla/mux (or chi router)
ORM	GORM
Database	PostgreSQL
Cache	Redis
Authentication	JWT (HS256, signed with JWT_SECRET)
Port	8080
Base URL	http://localhost:8080/api/v1

2. Data Models

2.1 User

Field	Type	Constraints
ID	UUID (string)	Primary key, auto-generated (UUIDv4)
Name	string	Required, max 100 characters
Email	string	Required, unique, valid email format
Password	string	Required, min 8 chars; stored as bcrypt hash
CreatedAt	timestamp	Auto-set on creation
UpdatedAt	timestamp	Auto-set on update

2.2 Application (App)

Field	Type	Constraints
ID	UUID (string)	Primary key, auto-generated
UserID	UUID (string)	Foreign key → User.ID
Name	string	Required, max 100 characters

Field	Type	Constraints
URL	string	Required, valid HTTPS URL, max 255 chars
RequestInterval	integer	Required; allowed values: 5, 10, or 15 (minutes)
IsEnabled	boolean	Default: false; must be explicitly enabled
CreatedAt	timestamp	Auto-set on creation
UpdatedAt	timestamp	Auto-set on update

2.3 RequestTimeFrame (RTF)

Field	Type	Constraints
ID	UUID (string)	Primary key, auto-generated
AppID	UUID (string)	Foreign key → Application.ID
StartTime	string	HH:MM (24-hr format), e.g. "06:00"
EndTime	string	HH:MM (24-hr format), must be after StartTime
Timezone	string	Valid IANA timezone, e.g. "Africa/Kampala"
CreatedAt	timestamp	Auto-set on creation
UpdatedAt	timestamp	Auto-set on update

2.4 Request (Ping Log)

Field	Type	Constraints
ID	UUID (string)	Primary key, auto-generated
AppID	UUID (string)	Foreign key → Application.ID
StatusCode	integer	HTTP status code received from pinged endpoint
Duration	integer	Round-trip time in milliseconds
MadeAt	timestamp	Timestamp when the ping was sent

3. Real Test Fixtures

The following concrete values **must** be used in all generated tests. Do not substitute generic placeholders.

3.1 Test Users

Field	Test User 1 (Owner)	Test User 2 (Intruder)
Name	Alice Nakato	Bob Okello
Email	alice.nakato@gmail.com	bob.okello@gmail.com
Password	Secure@1234	BobPass@5678
ID	6ba7b810-9dad-11d1-80b4-00c04fd430c8	7fa3c920-1bef-22e2-91c5-11d15fe541d9

3.2 Test Application

Field	Value
ID	3d6f4890-7b21-41d5-b2d3-4f9e8c6a1b2d
UserID	6ba7b810-9dad-11d1-80b4-00c04fd430c8 (Alice)
Name	My Render API
URL	https://my-api.onrender.com/active
RequestInterval	10 (minutes)
IsEnabled	false (default on creation)

3.3 Test Request Time Frame

Field	Value
ID	a1b2c3d4-e5f6-7890-abcd-ef1234567890
AppID	3d6f4890-7b21-41d5-b2d3-4f9e8c6a1b2d
StartTime	06:00
EndTime	23:00
Timezone	Africa/Kampala

3.4 Invalid / Not-Found IDs

Scenario	Value
Non-existent App ID	00000000-0000-0000-0000-000000000000

Scenario	Value
Non-existent RTF ID	ffffffff-ffff-ffff-ffffffffffff
Malformed UUID	not-a-valid-uuid
Wrong-user App ID	3d6f4890-7b21-41d5-b2d3-4f9e8c6a1b2d (access as Bob)

3.5 Environment Variables

Variable	Test Value
APPCRONS_TEST_DSN	host=localhost user=postgres password=test123 dbname=appcrons_test port=5432 sslmode=disable
REDIS_URL	redis://localhost:6379
JWT_SECRET	supersecretjwtkey1234567890abcdef

4. API Endpoints

4.1 Authentication (Public Endpoints)

Authentication endpoints are public — no JWT required. On success, a JWT token is returned which must be passed as **Authorization: Bearer <token>** on all protected routes.

POST /api/v1/auth/signup

Register a new user account.

Request Body:

```
{
  "name": "Alice Nakato",
  "email": "alice.nakato@gmail.com",
  "password": "Secure@1234"
}
```

Success Response:

```
{
  "status": "success",
  "data": {
    "user": { "id": "6ba7b810-9dad-11d1-80b4-00c04fd430c8",
    "name": "Alice Nakato", "email": "alice.nakato@gmail.com" },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}
```

Edge Cases:

- Missing name field → 400, validation error
- Missing email field → 400, validation error
- Missing password field → 400, validation error
- Empty request body → 400
- Invalid email format "notanemail" → 400
- Duplicate email "alice.nakato@gmail.com" → 409, "user with this email already exists"
- Password shorter than 8 chars (e.g. "abc") → 400
- SQL injection in email field → 400, sanitised
- Name longer than 100 characters → 400
- Name with only whitespace → 400

POST /api/v1/auth/signin

Authenticate an existing user and receive a JWT token.

Request Body:

```
{
  "email": "alice.nakato@gmail.com",
  "password": "Secure@1234"
}
```

Success Response:

```
{
  "status": "success",
  "data": {
    "user": { "id": "6ba7b810-9dad-11d1-80b4-00c04fd430c8",
      "name": "Alice Nakato", "email": "alice.nakato@gmail.com" },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}
```

Edge Cases:

- Correct credentials → 200 with valid JWT
- Unregistered email "unknown@gmail.com" → 401, "invalid credentials"
- Correct email, wrong password "WrongPass99" → 401, "invalid credentials"
- Missing email field → 400
- Missing password field → 400
- Empty string email "" → 400
- Empty string password "" → 400
- Email with leading/trailing whitespace → 400 or normalised
- Successful signin after signup → 200 with distinct JWT each time

4.2 Applications (Protected)

All application endpoints require the **Authorization: Bearer <token>** header. Users can only access their own applications.

POST /api/v1/apps — Create Application

Add a new application for automated ping monitoring.

Request Body:

```
{
  "name": "My Render API",
  "url": "https://my-api.onrender.com/active",
  "requestInterval": 10
}
```

Success Response:

```
{
  "status": "success",
  "data": { "app": {
    "id": "3d6f4890-7b21-41d5-b2d3-4f9e8c6a1b2d",
    "userId": "6ba7b810-9dad-11d1-80b4-00c04fd430c8",
    "name": "My Render API",
    "url": "https://my-api.onrender.com/active",
    "requestInterval": 10, "isEnabled": false
  }}
}
```

Edge Cases:

- No Authorization header → 401 Unauthorized
- Expired JWT token → 401 Unauthorized
- Malformed JWT → 401 Unauthorized
- Missing name → 400 validation error
- Missing url → 400 validation error
- Missing requestInterval → 400 validation error
- URL without HTTPS e.g. "http://myapi.onrender.com/active" → 400
- Invalid URL "notauri" → 400
- requestInterval = 7 (not in [5, 10, 15]) → 400
- requestInterval = 0 → 400
- requestInterval as string "ten" → 400 type error
- Name longer than 100 characters → 400
- Empty name "" → 400
- Newly created app must have isEnabled = false

GET /api/v1/apps — List Applications

Returns all applications owned by the authenticated user.

Success Response:

```
{ "status": "success", "data": { "apps": [ ... ] } }
```

Edge Cases:

- No Authorization header → 401

- Valid token, user has no apps → 200 with empty array []
- Valid token, user has 1+ apps → 200 with full list
- Bob's token → returns only Bob's apps, never Alice's
- Expired token → 401

GET /api/v1/apps/:id — Get Application

Real fixture ID: 3d6f4890-7b21-41d5-b2d3-4f9e8c6a1b2d

Edge Cases:

- Valid ID, correct owner (Alice) → 200 with app data
- Valid ID, accessed as different user (Bob) → 403 Forbidden
- Non-existent ID "00000000-0000-0000-0000-000000000000" → 404 Not Found
- Malformed UUID "not-a-valid-uuid" → 400 or 404
- No Authorization header → 401
- Expired token → 401

PATCH /api/v1/apps/:id — Update Application

Update one or more fields of an existing application.

Request Body:

```
{ "name": "Updated API", "url": "https://my-api.onrender.com/ping", "requestInterval": 15 }
```

Edge Cases:

- Valid update by owner → 200 with updated app
- Update only name, leave other fields unchanged → 200
- requestInterval = 20 (invalid) → 400
- Invalid URL format → 400
- Empty body → 200 (no-op) or 400 depending on implementation
- App ID not found → 404
- App owned by Bob, updated as Alice → 403
- No Authorization header → 401

DELETE /api/v1/apps/:id — Delete Application

Permanently delete an application and its associated data.

Edge Cases:

- Valid delete by owner → 200 or 204
- App ID not found → 404
- App owned by another user → 403
- Deleting same app twice → 404 on second attempt
- Cascading delete: associated RTFs and ping logs removed
- No Authorization header → 401

PATCH /api/v1/apps/:id/enable — Enable Application

Enable the application so it starts receiving automated pings.

Edge Cases:

- App currently disabled → 200, isEnabled: true
- App already enabled (idempotent) → 200, isEnabled: true
- App not found → 404
- App owned by another user → 403
- No Authorization header → 401

PATCH /api/v1/apps/:id/disable — Disable Application

Disable the application to stop automated pings.

Edge Cases:

- App currently enabled → 200, isEnabled: false
- App already disabled (idempotent) → 200, isEnabled: false
- App not found → 404
- App owned by another user → 403
- No Authorization header → 401

4.3 Request Time Frames (RTF)

RTFs define the daily window during which an application receives automated pings. All RTF endpoints are protected.

POST /api/v1/apps/:appId/request-time-frames — Create RTF

Add a time frame during which the application should receive pings.

Request Body:

```
{
  "startTime": "06:00",
  "endTime": "23:00",
  "timezone": "Africa/Kampala"
}
```

Success Response:

```
{
  "status": "success",
  "data": { "requestTimeFrame": {
    "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "appId": "3d6f4890-7b21-41d5-b2d3-4f9e8c6a1b2d",
    "startTime": "06:00", "endTime": "23:00",
    "timezone": "Africa/Kampala"
  }}
}
```

Edge Cases:

- Valid body → 201 Created
- startTime after endTime (e.g. "23:00" start, "06:00" end) → 400
- startTime equal to endTime (e.g. both "12:00") → 400
- Invalid time format (e.g. "6am" or "25:00") → 400
- Invalid IANA timezone "Mars/Olympus" → 400
- Missing startTime → 400
- Missing endTime → 400
- Missing timezone → 400
- App ID not found → 404
- App owned by another user (Bob accesses Alice's app) → 403
- No Authorization header → 401
- Expired JWT → 401

GET /api/v1/apps/:appId/request-time-frames — List RTFs

Returns all RTFs configured for the specified application.

Edge Cases:

- App has RTFs → 200 with array of RTF objects
- App has no RTFs → 200 with empty array []
- App not found → 404
- App owned by another user → 403
- No Authorization header → 401

PATCH /api/v1/apps/:appId/request-time-frames/:rtfId — Update RTF

Modify the time frame, timezone, or both for an existing RTF.

Request Body:

```
{ "startTime": "07:00", "endTime": "22:00", "timezone": "Africa/Nairobi" }
```

Edge Cases:

- Valid update by app owner → 200 with updated RTF
- startTime >= endTime → 400
- Invalid timezone → 400
- RTF not found → 404
- RTF belongs to a different app → 404
- App owned by another user → 403
- No Authorization header → 401

DELETE /api/v1/apps/:appId/request-time-frames/:rtfId — Delete RTF

Remove a time frame from an application.

Edge Cases:

- RTF exists and belongs to app → 200 or 204
- RTF not found → 404
- RTF ID belongs to a different app → 404
- App owned by another user → 403
- Deleting same RTF twice → 404 on second attempt
- No Authorization header → 401

4.4 Request Logs (Ping History)

GET /api/v1/apps/:appId/requests — List Ping Logs

Returns the history of all automated HTTP pings sent to an application.

Success Response:

```
{
  "status": "success",
  "data": { "requests": [
    { "id": "req_001", "appId": "3d6f4890-7b21-41d5-b2d3-4f9e8c6a1b2d",
      "statusCode": 200, "duration": 342, "madeAt": "2026-04-14T07:00:00Z" },
    { "id": "req_002", "appId": "3d6f4890-7b21-41d5-b2d3-4f9e8c6a1b2d",
      "statusCode": 503, "duration": 1200, "madeAt": "2026-04-14T07:10:00Z" }
  ]}
}
```

Edge Cases:

- App has ping logs → 200 with array of request objects
- App has no logs yet (newly created) → 200 with empty array []
- App not found → 404
- App owned by another user → 403
- No Authorization header → 401
- Expired JWT → 401
- Logs include failed pings (503) alongside successful (200) — both included in response

5. Authentication & Security

Rule	Detail
Token type	JWT, algorithm HS256
Token location	Authorization: Bearer <token> header
Token secret	JWT_SECRET environment variable
Token expiry	Implementation-defined; test with expired tokens
Missing header	401 Unauthorized
Bad/tampered token	401 Unauthorized
Expired token	401 Unauthorized
Cross-user access	403 Forbidden (user can only access own resources)

6. HTTP Status Code Reference

Code	Meaning	When Used
200	OK	Successful GET, PATCH, or DELETE
201	Created	Successful POST (resource created)
204	No Content	Successful DELETE with no body
400	Bad Request	Validation failure, malformed input, invalid field values
401	Unauthorized	Missing, expired, or invalid JWT token
403	Forbidden	Authenticated but accessing another user's resource
404	Not Found	Resource ID does not exist
409	Conflict	Duplicate unique field (e.g. duplicate email on signup)
500	Internal Server Error	Unexpected server-side failure

7. Background Scheduler Behaviour

The scheduler is a background goroutine that evaluates all enabled applications and sends HTTP GET pings to their configured URL endpoints.

- Ping frequency matches the application's requestInterval (5, 10, or 15 minutes).
- If the application has one or more RTFs, pings are sent only if the current time falls within any active RTF window (in the RTF's configured timezone).
- If no RTFs are configured, pings run 24 hours a day.

- Each ping result (status code, duration, timestamp) is stored in the Request (ping log) table.
- Disabled applications (isEnabled = false) are skipped entirely by the scheduler.
- Redis is used for distributed locking to avoid duplicate pings in multi-instance deployments.

8. Standard Error Response Format

All error responses follow this structure:

```
{
  "status": "error",
  "message": "human-readable error description",
  "errors": [ ... ] // optional array of field-level validation errors
}
```

TestSprite should assert both the HTTP status code **and** the **status: "error"** field for every negative test case.

9. TestSprite Integration Checklist

Required Checklist for Test Generation

- Use real fixture UUIDs, emails, and passwords from Section 3 — never generic placeholders.
- Test every HTTP method on every endpoint: happy path + all listed edge cases.
- Include auth tests (missing token, expired token, wrong-user) on every protected route.
- Assert HTTP status code AND response body fields in each test.
- Seed the database with Test User 1, Test User 2, and the Test App before each test run.
- Test idempotency for enable/disable endpoints.
- Test cascading deletes: deleting an app must remove its RTFs and ping logs.
- Verify isEnabled = false on all newly created apps.
- Verify RTF validation: startTime must strictly precede endTime.
- Verify all timezone strings are valid IANA timezones.